

Assignment II - SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- **Name:** 賴妤柔
- **Student ID:** 1101518
- **Course:** CS351
- **Date:** 2026/05/24

1. Introduction

近年來，AI-assisted software development 已逐漸成為軟體開發的重要趨勢。透過 LLMs 與 AI Coding Tools，開發者可以利用自然語言快速生成程式碼、測試案例、文件以及其他軟體開發相關產出，大幅提升開發效率與生產力。

然而，AI 並不會自動理解開發者真正的需求。如果需求描述不夠明確，AI 可能產生與預期不符的結果，甚至生成錯誤或不完整的程式碼。因此，在使用 AI 工具時，清楚且完整的需求定義變得格外重要。開發者需要提供明確的目標、功能需求、輸入輸出條件以及驗收標準，才能有效引導 AI 產生符合需求的結果。

在 AI 時代中，SDD、BDD 與 TDD 分別扮演不同但互補的角色。SDD

（Specification-Driven Development）透過明確的規格文件協助 AI 理解系統需求；BDD（Behavior-Driven Development）利用具體情境描述系統在不同條件下應有的行為；TDD（Test-Driven Development）則透過測試案例驗證系統是否正確實現需求。三者共同協助開發者降低需求誤解、提升溝通效率，並確保 AI 生成的結果符合預期。

2. Definition of SDD

SDD 是一種以規格為核心的軟體開發方法。在傳統開發流程中，規格文件多半只是提供開發人員參考；但在 AI 輔助軟體開發的環境下，規格的重要性進一步提升，甚至成為驅動程式生成的主要依據。

SDD 強調在實作前先釐清目標，並以清楚完整的規格文件描述問題、功能需求、輸入/輸出、限制與驗收條件作為開發與驗證依據，減少溝通與實作的轉譯落差。

近年來，隨著生成式 AI 的發展，SDD 也被部分開發者視為工程師版的 Vibe Coding。不同於僅依靠自然語言描述需求，SDD 強調使用標準化且完整的規格來引導 AI 產生程式碼，並在規格中預先定義程式品質、測試標準及效能需求。透過這種方式，AI 能更準確地理解開發者的意圖，進而產生更符合需求的軟體成果。

3. SDD: Student Grade Calculator

3.1 Goal

本系統的目標是根據學生的作業、期中考、期末考及專題成績，自動計算加權後的最終成績，並依照既定評分標準給予對應的等第成績。系統應提供一致且可靠的成績計算結果，以減少人工計算錯誤，並確保評分過程符合預先定義的規則。

3.2 Functional Requirements

FR-1：成績輸入功能

系統應允許使用者輸入作業、期中考、期末考及專題四項成績。

FR-2：資料驗證功能

系統應檢查所有輸入成績是否介於 0 至 100 分之間，若輸入不符合規定，應回傳錯誤訊息。

FR-3：成績計算功能

系統應根據各項成績的權重計算學生的最終成績，其中作業佔 30%、期中考佔 20%、期末考佔 30%、專題佔 20%。

FR-4：成績格式化功能

系統應將最終成績四捨五入至小數點後一位後再進行顯示。

FR-5：等第判定功能

系統應依據預先定義的成績區間規則，將最終成績轉換為對應的等第成績（A、B、C、D 或 F）。

FR-6：結果顯示功能

系統應顯示最終成績及其對應的等第成績，供使用者查閱。

3.3 Input

本系統需要接收四項成績作為輸入資料，所有成績皆應為 0 至 100 分之間的數值。

輸入項目	說明	有效範圍
Assignment	學生作業成績	0 ~ 100

輸入項目	說明	有效範圍
Midterm Exam	學生期中考成績	0 ~ 100
Final Exam	學生期末考成績	0 ~ 100
Project	學生專題成績	0 ~ 100

所有輸入資料皆須為數值型態，若輸入值超出有效範圍或非數值，系統應視為無效輸入並進行錯誤處理。

3.4 Output

本系統應根據輸入的成績資料產生以下輸出結果：

輸出項目	說明
Final Score	依照各項成績權重計算後的總成績，並四捨五入至小數點後一位
Letter Grade	根據最終成績所對應的等第（A、B、C、D 或 F）
Error Message	當輸入資料不符合規定時，顯示對應的錯誤訊息

系統應在輸入資料有效的情況下顯示最終成績與等第成績；若輸入資料超出允許範圍或格式錯誤，則應顯示錯誤訊息並停止成績計算。

3.5 Grade Rules

本系統應根據學生的 Final Score 判定對應的 Letter Grade。在進行等第判定前，系統應先將最終成績四捨五入至小數點後一位，再依照下列規則進行評分。

最終成績區間	等第成績
$90.0 \leq \text{Final Score} \leq 100.0$	A
$80.0 \leq \text{Final Score} < 90.0$	B
$70.0 \leq \text{Final Score} < 80.0$	C
$60.0 \leq \text{Final Score} < 70.0$	D
$\text{Final Score} < 60.0$	F

系統應確保每位學生僅能對應到一個等第成績，且所有等第判定皆須符合上述區間規則。

3.6 Acceptance Criteria

為確保 Student Grade Calculator 符合需求規格，系統應滿足以下驗收條件：

AC-1：正確計算最終成績

系統應依照指定權重正確計算最終成績，並將結果四捨五入至小數點後一位後再進行顯示。

AC-2：正確判定等第成績

系統應依照評分規則正確判定等第成績，且每位學生僅能對應一個等第。

AC-3：驗證輸入資料有效性

系統應驗證所有輸入資料的有效性。當輸入非數值資料，或任一成績超出 0 至 100 的範圍時，系統應顯示錯誤訊息並停止成績計算。

4. Definition of BDD

BDD 是一種以使用者行為為核心的軟體開發方法。它透過描述使用者在特定情境下的操作與系統應有的反應，來定義軟體需求。與傳統僅以技術規格或程式碼為主的方式相比，BDD 更重視從使用者角度理解系統應如何運作。

BDD 通常採用 **Given-When-Then** 結構來描述使用情境。其中，**Given** 表示前置狀態；**When** 表示使用者執行的動作；**Then** 表示系統應產生的結果。這種結構能將需求轉化為具體且容易理解的情境，使需求描述更加清楚。

BDD 有助於明確表達系統需求與預期結果，因為它會將需求轉換成具體的使用情境。例如，與其只寫「系統必須計算成績」，BDD 會進一步描述使用者在特定情境下的操作，以及系統應產生的結果。透過這種方式，所有參與者都能清楚知道在特定情況下系統應該如何運作，而不需要理解程式碼或技術細節。因此，BDD 能有效減少需求理解上的差異，讓開發結果更符合使用者期待。

5. BDD: Student Grade Calculator

```
Scenario 1: Student receives grade B
Given the assignment score is 85
And the midterm exam score is 78
And the final exam score is 82
And the project score is 90
When the system calculates the final grade
Then the final score should be 84.0
And the letter grade should be B
```

```
Scenario 2: Student receives grade A at the boundary
Given the assignment score is 90
And the midterm exam score is 90
And the final exam score is 90
And the project score is 90
When the system calculates the final grade
```

```
Then the final score should be 90.0
And the letter grade should be A
```

```
Scenario 3: Student enters an invalid score
Given the assignment score is 88
And the midterm exam score is 75
And the final exam score is 102
And the project score is 80
When the system validates the input scores
Then the system should display an invalid input message
And the system should not calculate the final score
```

6. Definition of TDD

TDD 是一種以測試為核心的軟體開發方法。TDD 強調先根據需求撰寫測試案例，再實作對應的功能，最後進程式碼重構（Refactoring）。整個開發過程通常遵循「Red-Green-Refactor」循環：先撰寫測試案例並執行測試，由於功能尚未完成，因此測試通常會失敗（Red），接著撰寫程式碼使測試通過（Green），最後在不改變系統行為的前提下持續改善程式碼結構與品質（Refactor）。

TDD 的核心目標是透過測試來驅動系統設計與開發。在實作功能之前，開發者必須先思考系統應該產生什麼結果以及如何驗證其正確性，因此能夠更清楚地理解需求並降低設計錯誤的風險。

TDD 有助於提升軟體品質，因為每個功能都會搭配對應的測試案例進行驗證。當程式碼修改或新增功能時，既有測試可以快速檢查系統是否受到影響，降低缺陷被引入的可能性。此外，由於開發者需要持續重構程式碼並保持所有測試通過，因此能夠維持程式碼的可讀性、可維護性與穩定性，進而提高整體軟體品質。

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Test Case 1: Calculate grade B

Input

- Assignment: 85
- Midterm Exam: 78
- Final Exam: 82
- Project: 90

Expected Calculation

- Final Score = $85 \times 0.30 + 78 \times 0.20 + 82 \times 0.30 + 90 \times 0.20$
- Final Score = $25.5 + 15.6 + 24.6 + 18.0$
- Final Score = 83.7

Expected Output

- Final Score: 83.7
- Letter Grade: B

Test Case 2: Calculate grade C

Input

- Assignment: 72
- Midterm Exam: 68
- Final Exam: 75
- Project: 80

Expected Calculation

- Final Score = $72 \times 0.30 + 68 \times 0.20 + 75 \times 0.30 + 80 \times 0.20 = 21.6 + 13.6 + 22.5 + 16.0 = 73.7$

Expected Output

- Final Score: 73.7
- Letter Grade: C

Scenario 2: Boundary Test Cases

Test Case 1: Boundary score for grade A

Input

- Assignment: 90
- Midterm Exam: 90
- Final Exam: 90
- Project: 90

Expected Calculation

Final Score = $90 \times 0.30 + 90 \times 0.20 + 90 \times 0.30 + 90 \times 0.20 = 27.0 + 18.0 + 27.0 + 18.0 = 90.0$

Expected Output

- Final Score: 90.0
- Letter Grade: A

Test Case 2: Boundary score for grade D

Input

- Assignment: 60
- Midterm Exam: 60
- Final Exam: 60
- Project: 60

Expected Calculation

Final Score = $60 \times 0.30 + 60 \times 0.20 + 60 \times 0.30 + 60 \times 0.20 = 18.0 + 12.0 + 18.0 + 12.0 = 60.0$

Expected Output

- Final Score: 60.0
- Letter Grade: D

Scenario 3: Invalid Input Test Cases

Test Case 1: Score below valid range

Input

- Assignment: -5
- Midterm Exam: 80
- Final Exam: 75
- Project: 90

Expected Calculation

The system should not calculate the final score because the assignment score is below 0.

Expected Output

- Final Score: Not calculated
- Letter Grade: Not assigned
- Error Message: Invalid input. Scores must be numbers between 0 and 100.

Test Case 2: Score above valid range

Input

- Assignment: 88
- Midterm Exam: 76
- Final Exam: 105
- Project: 82

Expected Calculation

The system should not calculate the final score because the final exam score is greater than 100.

Expected Output

- Final Score: Not calculated
- Letter Grade: Not assigned
- Error Message: Invalid input. Scores must be numbers between 0 and 100.

8. Comparison of SDD, BDD, and TDD

項目	SDD	BDD	TDD
全名	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
核心重點	需求與系統規格	系統行為與使用情境	測試與功能驗證
主要問題	系統應該實現什麼功能？	系統在特定情境下應如何運作？	如何確認系統功能正確？
主要產出	規格文件	行為情境	測試案例
常見格式	Goal、Requirements、Input、Output、	Given-When-Then	Input、Expected Output、Test

項目	SDD	BDD	TDD
	Acceptance Criteria		Result
主要對象	開發者、需求提出者	開發者、需求提出者、非技術人員	開發者與測試人員
Student Grade Calculator 範例	定義成績規則、輸入輸出與驗收條件	描述使用者輸入成績後系統應有的反應	驗證正常、邊界與錯誤輸入案例
AI 時代價值	協助 AI 理解需求與規格	協助 AI 理解預期行為與情境	驗證 AI 產生的結果是否正確
主要優點	降低需求誤解，建立明確開發目標	提升溝通效率，建立共同理解	提高程式品質，降低缺陷風險

9. Reflection

在 SDD、BDD 與 TDD 三種方法中，我覺得最容易理解的是 TDD。因為它和我修程式設計時的作業模式很像。老師通常會先定義好需要完成的 function，並提供一系列測試資料，最後再根據程式通過測試的程度來評分。因此在接觸 TDD 時，我很快就能理解它的概念。

如果要選出一個最適合搭配 AI Coding 工具的方法，我會選擇 SDD。因為 AI 本身並不知道使用者真正想要什麼，如果只給它一句模糊的需求，很容易產生與預期不符的結果。但當規格都被明確定義後，AI 就能更容易理解需求並生成對應的程式碼。

BDD 則讓我看到另一個角度。相比於規格本身，BDD 更重視使用者實際會遇到哪些情境。透過 Given-When-Then 的方式，可以更具體地描述使用者的操作以及系統應有的反應。我認為這種方法更適合與非技術人員溝通，因為他們不需要理解程式碼，也能知道系統是否符合自己的期待。

至於 TDD，我認為它最大的價值在於驗證。無論程式碼是自己寫的還是 AI 生成的，都需要透過測試來確認結果是否正確。透過正常案例、邊界案例以及錯誤輸入案例，可以更全面地檢查程式品質，而不只是確認它在理想情況下能夠執行。

如果未來要在專案中結合 AI 工具，我會先使用 SDD 建立完整規格，再利用 TDD 設計測試案例。對於單人專案而言，這樣通常已經足夠。但如果專案涉及產品經理、客戶或其他非技術人員，我會額外使用 BDD 來描述系統行為，協助團隊建立共同認知。

10. References / AI Tool Usage

- 使用工具：ChatGPT
- 使用內容：
 - 理解 SDD、BDD 與 TDD 概念

- 整理作業內容
 - 檢查報告結構與文字表達
 - 備註：
 - 所有 AI 產出內容皆經過個人檢查與修改。
-

- **規格驅動開發 (SDD)，又一個銀子彈？**

<https://yyllab.tw/what-is-the-spec-driven-development/>

- **從想法到程式碼：Specification-Driven Development 完全實戰指南**

<https://ithelp.ithome.com.tw/m/articles/10391813>

- **BDD/TDD差別是什麼？手把手用 Cucumber 實作示範BDD**

<https://medium.com/@onejar99/bdd-tdd差別是什麼-手把手用-cucumber-實作示範bdd-32ee61aed372>

- **TDD 開發五步驟，帶你實戰 Test-Driven Development 範例**

[https://tw.alphacamp.co/blog/tdd-test-driven-development-example#什麼是-tdd \(test-driven-development\) ?](https://tw.alphacamp.co/blog/tdd-test-driven-development-example#什麼是-tdd-(test-driven-development)-?)